Universidad del Valle
Escuela de Ingeniería de Sistemas y Computación

Taller 1: Desarrollo de un MVP usando Ingeniería de Software Aumentada con IA en el marco de la metodología ágil Scrum

Pipeline de desarrollo del MVP

Etapas, aplicación de la inteligencia artificial y resultados

Documento 1 de 3 · Describe cómo se desarrolló CampusConecta en cada etapa del pipeline y qué resultado se obtuvo.

Equipo de desarrollo

Gabriela Guzmán Botina · Desarrolladora Backend
Juliana Araujo · Desarrolladora Backend
Laura Stefania Salazar · Desarrolladora Frontend
Juan José Padilla · Despliegue y Desarrollador Backend
Daniel Trujillo · Desarrollador Frontend
Juan Sebastián Rodas Ramírez · Tester y Arquitecto de Software

Repositorio: https://github.com/gabrielaguzmn/CampusConecta
Cali, 13 de octubre de 2026

1. Introducción

CampusConecta es una plataforma web responsive que reúne en un solo lugar las actividades extracurriculares del campus de la Universidad del Valle: permite publicarlas con una o varias fechas, consultarlas en un feed con filtros, ubicarlas en un plano del campus y moderarlas a partir de reportes de la comunidad.

Este documento aplica al proyecto el pipeline de nueve etapas propuesto en el taller. Para cada etapa indica cómo se usó la inteligencia artificial, qué revisó o cambió el equipo y qué resultado se obtuvo. El estado de cada etapa usa esta escala:

Estado	Significado
Implementado y verificado	Existe en el código y las pruebas automatizadas lo comprueban.
Implementado, pendiente de verificación	Existe en el código, pero falta comprobar algún punto.
Parcialmente implementado	Cumple una parte de lo esperado.
Documentado	El resultado de la etapa es un documento o un plan revisado por el equipo.

El detalle de requisitos, historias de usuario, pruebas y trazabilidad está en la Documentación del MVP, y la representación gráfica del ciclo, en el Diagrama del pipeline.

2. Pipeline aplicado al desarrollo del MVP

Las columnas «Aplicación de IA» y «Resultado esperado» reproducen la tabla del taller; las dos últimas muestran lo obtenido en CampusConecta.

Etapa	Aplicación de IA	Resultado esperado	Resultado en CampusConecta	Estado
1. Descubrimiento	Analizar el problema, los usuarios y el alcance	Idea del producto y MVP	Documento de definición del MVP: problema, objetivos, usuarios, módulos, alcance y exclusiones.	Documentado
2. Requerimientos	Generar y revisar historias, criterios de aceptación	Product Backlog refinado	34 requisitos funcionales, 7 no funcionales y 20 historias de usuario con 68 criterios de aceptación.	Documentado
3. Planificación	Clasificar, priorizar y estimar historias	Sprint Backlog y plan de trabajo	Release Plan: 1 sprint de dos semanas, 77 Story Points, prioridades y responsables por historia.	Documentado
4. Arquitectura	Proponer componentes, modelo de datos, API y decisiones tecnológicas	Diseño de arquitectura	Mockups de alta fidelidad, arquitectura cliente-servidor, modelo de datos de 8 entidades y contrato de la API.	Implementado y verificado
5. Desarrollo	Generar código, explicar soluciones y realizar revisiones	Incrementos funcionales	Las 20 historias implementadas en una API REST y una aplicación web responsive.	Implementado, pendiente de verificación
6. Pruebas	Generar casos unitarios, integración, seguridad y aceptación	Conjunto de pruebas automatizadas	195 pruebas de backend y 153 de interfaz; todas pasan.	Parcialmente implementado
7. CI/CD	Crear y revisar el pipeline de integración y despliegue	Aplicación desplegada (local o en la nube)	Integración continua con GitHub Actions (pruebas y compilación en cada cambio); aplicación en local con base de datos en la nube; sin despliegue automático.	Parcialmente implementado
8. Documentación	Generar documentación técnica y manual de usuario	Documentación actualizada	Guía de instalación y ejecución, contrato de la API, documentación del MVP y manual de usuario.	Documentado
9. Evaluación	Analizar resultados del MVP y la calidad de las respuestas de IA	Informe de resultados y mejoras aplicadas	Evaluación de resultados y de la IA, con las mejoras aplicadas y las pendientes (sección 3.9).	Documentado

3. Desarrollo de cada etapa

3.1 Descubrimiento

Qué se hizo	El equipo partió del documento de formulación del proyecto, que describe el problema: la información sobre actividades extracurriculares del campus está dispersa entre grupos de WhatsApp, redes sociales y voz a voz, lo que reduce su visibilidad y la participación.
Uso de la IA	Un prompt con el rol de analista de requisitos transformó ese documento en una definición formal del MVP. El prompt prohibía inventar funcionalidades, pedía conservar el significado original y marcar como «Pendiente de definición» lo que faltara.
Revisión del equipo	El equipo completó la información que la IA señaló como pendiente en sucesivas versiones del documento, hasta fijar dos roles (usuario registrado y administrador), seis objetivos específicos y lo que queda fuera del alcance.
Estado	**Documentado**

3.2 Requerimientos

Qué se hizo	Se formalizaron los requisitos y se escribieron las historias de usuario del backlog.
Uso de la IA	El mismo prompt de análisis redactó los requisitos con la forma «El sistema deberá…». Después, un prompt con el rol de Scrum Master generó las historias con la estructura Como / Quiero / Para, criterios Dado / Cuando / Entonces y Definition of Done.
Revisión del equipo	Al desarrollar, la IA debía citar textualmente el requisito que respaldaba cada regla. Cuando no encontraba respaldo, registraba una pregunta para el equipo en lugar de suponer la respuesta. Así se detectaron y resolvieron vacíos como la visibilidad de una fecha cancelada o la regla de eliminación de eventos.
Estado	**Documentado**

3.3 Planificación

Qué se hizo	Se organizó el trabajo en un solo release con un sprint del 28 de septiembre al 13 de octubre de 2026.
Uso de la IA	El prompt de Scrum Master estimó cada historia en escala Fibonacci, la priorizó con MoSCoW, calculó un puntaje de uso de IA (de 0 a 5) y propuso un desarrollador principal y uno de apoyo, sin superar el 50 % de carga por persona.
Revisión del equipo	El prompt pedía tres épicas obligatorias; el equipo añadió Acceso, Categorías y Administración para cubrir todos los requisitos, y quedaron seis. También corrigió el número de integrantes, que el prompt indicaba como cinco aunque listaba seis.
Estado	**Documentado**

3.4 Arquitectura

Qué se hizo	Se diseñaron la interfaz y la arquitectura técnica antes de programar.
Uso de la IA	Un prompt con el rol de diseñador UI/UX generó los mockups de siete vistas (inicio de sesión, registro, recuperación, feed, mis eventos, ventana de gestión y mapa) con colores, tipografía y estados definidos. Los prompts de desarrollo pidieron después proponer el modelo de datos, los servicios de la API y los permisos, justificando cada decisión con los requisitos.
Revisión del equipo	El equipo aprobó la arquitectura antes de generar código. Entre las decisiones revisadas están calcular el estado de cada fecha en el momento de la consulta, una sola ubicación por evento y un catálogo fijo de lugares del campus.
Estado	**Implementado y verificado**

3.5 Desarrollo

Qué se hizo	Se construyó el producto por capas: modelo de datos, servicios de la API y pantallas.
Uso de la IA	Los prompts de backend y frontend generaron el código capa por capa. Cada capa debía terminar con sus pruebas pasando y esperar la confirmación del equipo antes de continuar.
Revisión del equipo	El equipo revisó cada capa y ajustó el resultado cuando los requisitos lo exigían. Por ejemplo, el mockup limitaba la ventana de gestión a tres acciones y se añadió «Más acciones» para finalizar, extender y cancelar todas las fechas. Queda por verificar el plano del mapa en un navegador real.
Estado	**Implementado, pendiente de verificación**

3.6 Pruebas

Qué se hizo	Se escribieron pruebas automatizadas para el backend y para la interfaz.
Uso de la IA	La IA generó pruebas unitarias y de integración nombradas con la historia y el criterio que verifican, incluidas pruebas de permisos (acceso denegado por rol, por propiedad y por estado).
Revisión del equipo	Desde el 13 de octubre de 2026 estas pruebas se ejecutan automáticamente en cada cambio (etapa 7). No se han hecho pruebas de aceptación con usuarios, de extremo a extremo en navegador ni de rendimiento; por eso la etapa queda parcialmente implementada.
Estado	**Parcialmente implementado**

3.7 CI/CD

Qué se hizo	El repositorio tiene un pipeline de integración continua en GitHub Actions. Se ejecuta en cada push, en cada pull request hacia main y de forma manual, con dos trabajos en paralelo. **Backend:** levanta PostgreSQL 16, instala las dependencias, verifica la configuración y ejecuta las pruebas. **Frontend:** instala las dependencias, ejecuta las pruebas, verifica los tipos y compila. La aplicación se ejecuta en local con una base de datos PostgreSQL compartida en la nube (Render); no hay despliegue automático.
Uso de la IA	El pipeline ejecuta las mismas verificaciones que los prompts de desarrollo exigían al cerrar cada capa. No se conserva registro de si su configuración se generó con IA.
Revisión del equipo	Antes del pipeline se integró código incompleto, por ejemplo cambios en el modelo de datos sin su migración. Ahora un cambio con pruebas fallidas queda marcado en GitHub. El pipeline detectó un error de formato en la recuperación de contraseña que bloqueó un pull request; se corrigió y, al reproducir sus pasos el 13 de octubre de 2026, ambos trabajos pasan.
Estado	**Parcialmente implementado**

3.8 Documentación

Qué se hizo	Se documentó la instalación, el uso y el diseño del sistema.
Uso de la IA	La IA redactó la guía de instalación y ejecución del repositorio (Linux y Windows), generó el contrato de la API desde el código y apoyó la redacción de la documentación del MVP, que incluye el manual de usuario.
Revisión del equipo	El equipo revisó los documentos frente al código y corrigió lo desactualizado, por ejemplo la base de datos compartida y la carga del catálogo de ubicaciones en la guía de instalación.
Estado	**Documentado**

3.9 Evaluación

Qué se hizo	Se evaluaron los resultados del MVP y la calidad de las respuestas de la IA.
Uso de la IA	La IA analizó los resultados de las pruebas, comparó la documentación con el código y señaló las diferencias encontradas.
Revisión del equipo	El equipo aplicó las mejoras de documentación y priorizó las pendientes para la siguiente iteración.
Estado	**Documentado**

Aspecto evaluado	Resultado
Funcionalidad del MVP	Las 20 historias implementadas; 195 de 195 pruebas de backend y 153 de 153 de interfaz pasan.
Aciertos de la IA	Exigir citas textuales de los requisitos evitó reglas inventadas y convirtió los vacíos en preguntas para el equipo.
Fallos detectados en la IA	Leyó nombres internos de capas del diseño en lugar del texto visible, partió de un archivo de diseño desactualizado y dejó mensajes de una librería en inglés. Todos se corrigieron en la revisión.
Mejoras aplicadas	Pipeline de integración continua y corrección del error que detectó, guía de instalación actualizada, contrato de la API regenerado y reglas de negocio aclaradas con el equipo.
Mejoras pendientes	Despliegue automático, pruebas de aceptación con usuarios y prueba del mapa en navegador.

4. Integración con Scrum

El pipeline se ejecutó dentro de un único sprint. Las etapas 1 a 3 produjeron el Product Backlog y el plan del sprint; las etapas 4 a 8 se recorrieron por cada historia (diseño, código, pruebas y documentación), y la etapa 9 corresponde al cierre del sprint. Las 20 historias tienen prioridad Must Have, y el puntaje de IA del Release Plan fue 0 de 5 (tipo tradicional) en todas, porque ninguna funcionalidad del producto usa IA: la IA se usó como herramienta de ingeniería, no dentro de la aplicación.

Integrante	Rol	SP principal	SP apoyo	Carga del release
Gabriela Guzmán Botina	Desarrolladora Backend	15	2,5	22,7 %
Juliana Araujo	Desarrolladora Backend	15	9	31,2 %
Laura Stefania Salazar	Desarrolladora Frontend	10	5,5	20,1 %
Juan José Padilla	Despliegue y Desarrollador Backend	15	5	26,0 %
Daniel Trujillo	Desarrollador Frontend	14	10	31,2 %
Juan Sebastián Rodas Ramírez	Tester y Arquitecto de Software	8	6,5	18,8 %

La carga cuenta el apoyo a la mitad de sus Story Points; nadie supera el 50 % del release. No se conservan registros de las reuniones del sprint (daily, review y retrospectiva); el avance se refleja en el historial de commits del repositorio.